AI-Powered Startup Due Diligence Platform — 12-Day Roadmap (Full Scope)

Student: Dhiren | UPES Dehradun **Assigned Stack:** FastAPI, Scikit-learn, XGBoost, LLMs, pdfplumber, Streamlit/React, PostgreSQL, Docker **Timeline:** 12 days, full scope (nothing cut)

---

1. Project Vision (one-liner)

A platform where a user uploads a startup's pitch deck (PDF), the system extracts key data points, an ML engine scores investment risk/success probability, and an LLM writes a human-readable due-diligence report — all shown on a dashboard.

---

2. Architecture (6 layers)

Layer	Tools	Purpose
1. Data Ingestion	pdfplumber	Read pitch deck PDFs / upload CSVs
2. Data Processing	Pandas, NumPy, Regex	Clean & structure extracted data
3. ML/AI Engine	Scikit-learn, XGBoost, OpenAI API	Predict risk score + generate narrative report
4. Backend	FastAPI	Serve predictions via API endpoints

Layer	Tools	Purpose
5. Database	PostgreSQL	Store startups, scores, reports
6. Frontend	Streamlit (MVP) + React (polish, if time allows)	Dashboard for uploading + viewing results
7. Deployment	Docker, Render/Railway	Host the live demo

---

3. Dataset Strategy (full)

- **Primary dataset:** Kaggle "Startup Success Prediction" (923 rows, 48 features, target = acquired/closed)
- **Enrichment:** Y Combinator company metadata (industry, batch, team size) merged in via company-name matching, for extra features
- **Honest framing for viva:** "We combine real startup outcome data with a synthetic/parsed financial layer, since real private-company financials are not publicly available — standard practice for academic due-diligence work."

---

4. Full Differentiation Set (all included, none cut)

1. **Live PDF parsing** — real pitch deck upload → auto-extracted data
2. **Explainability** — SHAP values / feature importance shown on dashboard
3. **LLM-generated narrative report** — 1-page due-diligence memo
4. **Multi-source data merge** — YC metadata + Kaggle financial dataset
5. **Sector-aware risk logic** — different risk factors per industry (e.g. FinTech regulatory risk vs SaaS churn risk)

---

5. Day-by-Day Plan

Day 1 — Setup

- Install VS Code, Python, Docker Desktop (done ✅)
- Create project folder structure, virtual environment, GitHub repo
- Install all dependencies (`requirements.txt`)
- Download Kaggle "Startup Success Prediction" dataset
- Download/prepare YC metadata dataset
- **Goal:** both datasets loading cleanly in Pandas

Day 2 — Data Cleaning + Merge

- Clean Kaggle dataset: missing values, encode categoricals
- Merge YC metadata in (company-name matching where possible)
- Full EDA: distributions, correlations, class balance
- **Goal:** single clean merged dataframe ready for feature engineering

Day 3 — Feature Engineering + Sector Logic

- Engineer features: growth ratios, funding velocity, team size buckets
- Design sector-specific risk flags (e.g. FinTech = regulatory flag, SaaS = churn-sensitivity flag) as extra columns
- **Goal:** final feature set locked, documented in a data dictionary

Day 4 — PDF Parser

- Build `pdfplumber` extraction script
- Build regex patterns for revenue, funding ask, team size, industry keywords
- Test on 3-4 real/sample pitch decks

- Map extracted fields to the same schema as the training data
- **Goal:** PDF → structured dict, matching model input format

Day 5 — ML Models (Round 1)

- Train Logistic Regression (baseline)
- Train Random Forest
- Train XGBoost
- Evaluate: accuracy, F1, ROC-AUC, confusion matrix
- **Goal:** first working models, comparison table drafted

Day 6 — ML Models (Round 2) + Explainability

- Hyperparameter tuning (GridSearch/RandomizedSearch) on best model
- Add SHAP explainability (or feature_importances_ if SHAP setup is heavy)
- Save final model + explainability artifacts with `joblib`
- **Goal:** tuned final model + explainability plots ready to plug into dashboard

Day 7 — LLM Report Layer

- Connect OpenAI API
- Design prompt template: structured data + score + top risk factors → narrative report (Risk Summary, Strengths, Red Flags, Recommendation)
- Test on 4-5 sample startups, refine prompt for consistent tone
- **Goal:** reliable LLM report generation function

Day 8 — Backend (FastAPI)

- Build endpoints:
  - `POST /upload-pitch-deck` → parses PDF
  - `POST /predict` → runs ML model, returns score + explainability

    - `POST /generate-report` → runs LLM, returns narrative
    - `GET /startups/{id}` → fetch stored record
    - `GET /startups` → list history
- Connect PostgreSQL via SQLAlchemy, create tables (startups, scores, reports)
- **Goal:** all endpoints tested and working via `/docs`

Day 9 — Frontend Dashboard (Streamlit MVP)

- Upload PDF → show extracted data (editable)
- Run prediction → show score + explainability chart
- Generate report → show LLM narrative
- History table of past evaluations
- **Goal:** full working demo end-to-end, Streamlit version

Day 10 — Frontend Polish (React, optional upgrade)

- If time/comfort allows: rebuild key screens in React for a more professional UI (upload, results, history)
- If not: polish the Streamlit UI (better layout, charts, colors)
- **Goal:** presentable, demo-ready UI either way

Day 11 — Docker + Deployment

- Write `Dockerfile`(s) + `docker-compose.yml` for backend, frontend, DB
- Test full stack locally with `docker-compose up`
- Deploy to Render or Railway
- Test the live link end-to-end (upload → score → report)
- **Goal:** live deployed demo link working

Day 12 — Report + Viva Prep

- Write final project report (problem statement, architecture, dataset, methodology, results, limitations, future work)
- Prepare slide deck if required
- Prepare viva script:
  - Model comparison justification
  - Dataset limitations & honest framing
  - Live demo walkthrough
  - Answer for "is this industry-level?" (see Section 7)
- **Goal:** fully submission-ready + rehearsed for viva

---

6. Deliverables Checklist

- [ ] GitHub repo with clean code structure
- [ ] Merged, cleaned dataset (Kaggle + YC)
- [ ] Tuned ML model with explainability
- [ ] Working PDF parser
- [ ] Working LLM report generator
- [ ] FastAPI backend (all endpoints documented)
- [ ] Streamlit dashboard (+ React polish if done)
- [ ] Dockerized app
- [ ] Live deployed demo link
- [ ] Final project report
- [ ] Viva script + slide deck

---

7. Viva-Ready Framing (memorize this)

> "We built a functional prototype using an industry-standard tech stack and architecture — similar to real fintech/due-diligence platforms — trained on publicly available startup outcome data, enriched with Y Combinator company metadata. Since real proprietary financials aren't publicly accessible, we combine real outcome labels with a parsed/synthetic financial layer. Moving this to production would additionally require verified data sources, security, and compliance work."

Next step: build the actual code, starting with Day 1 — project setup and dataset loading.